

Python Application Deployment on Linux

Documentation & Best Practices

By Dr. D.S.R. Murthy

1. Server Preparation

1.1 Base setup

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y build-essential python3-dev python3-venv python3-pip \
```

```
git curl nginx supervisor ufw fail2ban
```

1.2 Create a dedicated, non-root deploy user

Never run the application as root.

```
sudo adduser deploy --disabled-password
```

```
sudo usermod -aG sudo deploy # only if this user needs admin rights
```

1.3 Firewall basics

```
sudo ufw allow OpenSSH
```

```
sudo ufw allow 'Nginx Full'
```

```
sudo ufw enable
```

2. Python Version & Environment Management

- Use **pyenv** if you need a Python version different from the OS default, or if multiple apps on the box need different versions.

- Always isolate dependencies with a **virtual environment** — never install app dependencies into the system Python.

```
python3 -m venv /home/deploy/apps/myapp/venv
```

```
source /home/deploy/apps/myapp/venv/bin/activate
```

```
pip install --upgrade pip wheel
```

Dependency pinning

- requirements.txt for simple apps; pin exact versions (==) for reproducibility.
- Prefer **Poetry** or **pip-tools** (pip-compile) for larger projects — they generate a lock file separate from the abstract dependency list.
- Never deploy with unpinned dependencies (>= only) — this breaks reproducible builds.

```
pip install -r requirements.txt
```

or

```
poetry install --no-dev
```

3. Application Layout (recommended convention)

```
/home/deploy/apps/myapp/
```

```
|— venv/
```

```
|— src/          # application code
```

```
|— requirements.txt
```

```
|— .env          # secrets (never committed to git)
```

```
|— gunicorn_conf.py
```

|— logs/

|— manage.py / wsgi.py / asgi.py

Keep code, virtual environment, logs, and config **separate** but under one app root. Use git to deploy (pull-based) or a CI artifact (build-then-ship) — avoid editing code directly on the server.

4. Configuration & Secrets

- **Never hardcode secrets** (DB passwords, API keys) in source code.
 - Store secrets in environment variables, loaded via `.env` (with `python-dotenv`) or injected by the process manager / orchestrator.
 - Restrict permissions on `.env`:
 - `chmod 600 .env`
 - For anything beyond a single server, use a secrets manager: HashiCorp Vault, AWS Secrets Manager, or `systemd`'s `EnvironmentFile=`.
 - Keep `DEBUG=False` in production always — a Django/Flask debug page leaking in prod is a common, serious security hole.
-

5. Application Server (WSGI/ASGI)

Python's built-in dev server (`flask run`, `manage.py runserver`) is **not** production-safe — no concurrency control, no process management, weak security defaults.

Framework type	Recommended server
WSGI (Flask, Django sync)	Gunicorn

Framework type	Recommended server
ASGI (FastAPI, Django async, Starlette)	Uvicorn (often behind <i>Gunicorn</i> as a process manager: <code>gunicorn -k uvicorn.workers.UvicornWorker</code>)

Example Gunicorn config (`gunicorn_conf.py`)

```
bind = "127.0.0.1:8000"
```

```
workers = 4          # ~ (2 x CPU cores) + 1
```

```
worker_class = "sync"  # or "uvicorn.workers.UvicornWorker" for ASGI
```

```
timeout = 30
```

```
graceful_timeout = 30
```

```
max_requests = 1000    # recycle workers to avoid memory leaks
```

```
max_requests_jitter = 50
```

```
accesslog = "/home/deploy/apps/myapp/logs/access.log"
```

```
errorlog = "/home/deploy/apps/myapp/logs/error.log"
```

```
loglevel = "info"
```

Run it manually to test:

```
venv/bin/gunicorn -c gunicorn_conf.py src.wsgi:application
```

Never expose Gunicorn/Uvicorn directly to the internet — bind it to 127.0.0.1 and put a reverse proxy in front.

6. Process Management (keep it running & auto-restart)

Use **systemd** (preferred on modern Linux) instead of screen/tmux/nohup.

/etc/systemd/system/myapp.service:

[Unit]

Description=Gunicorn instance for myapp

After=network.target

[Service]

User=deploy

Group=www-data

WorkingDirectory=/home/deploy/apps/myapp

EnvironmentFile=/home/deploy/apps/myapp/.env

ExecStart=/home/deploy/apps/myapp/venv/bin/gunicorn -c gunicorn_conf.py
src.wsgi:application

Restart=always

RestartSec=5

StandardOutput=append:/home/deploy/apps/myapp/logs/stdout.log

StandardError=append:/home/deploy/apps/myapp/logs/stderr.log

[Install]

WantedBy=multi-user.target

sudo systemctl daemon-reload

```
sudo systemctl enable --now myapp
```

```
sudo systemctl status myapp
```

```
sudo journalctl -u myapp -f    # live logs
```

(Supervisor is a valid alternative if you prefer a Python-native tool, but systemd is the modern standard and gives OS-level integration, cgroups, and log control.)

7. Reverse Proxy (Nginx)

Nginx handles TLS termination, static files, request buffering, and load distribution — Gunicorn/Uvicorn should never face the raw internet.

/etc/nginx/sites-available/myapp:

```
server {  
  
    listen 80;  
  
    server_name myapp.example.com;  
  
    location /static/ {  
  
        alias /home/deploy/apps/myapp/staticfiles/;  
  
    }  
  
    location / {  
  
        proxy_pass http://127.0.0.1:8000;  
  
        proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;

proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

proxy_set_header X-Forwarded-Proto $scheme;

proxy_redirect off;

}

}
```

```
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/
```

```
sudo nginx -t
```

```
sudo systemctl reload nginx
```

TLS

```
sudo apt install certbot python3-certbot-nginx
```

```
sudo certbot --nginx -d myapp.example.com
```

Certbot auto-configures renewal via a systemd timer/cron — verify with `sudo certbot renew --dry-run`.

8. Database & Migrations

- Never run migrations from a developer's laptop against prod — run them as part of the deploy pipeline, with a backup taken first.
- Use connection pooling (pgbouncer for Postgres) if you have many worker processes.
- Store DB credentials in `.env` / secrets manager, not in `settings.py`.

```
venv/bin/python manage.py migrate --noinput
```

9. Logging & Monitoring

- Application logs → files under the app's logs/ dir, rotated with **logrotate**:
 - `/home/deploy/apps/myapp/logs/*.log { daily rotate 14 compress missingok notifempty}`
 - System/service logs → `journalctl -u myapp`.
 - Add health-check endpoints (`/healthz`) for uptime monitors / load balancers.
 - For real production visibility, ship logs and metrics to something like Prometheus + Grafana, Sentry (error tracking), or a hosted APM (Datadog, New Relic).
-

10. Deployment Workflow

1. **Build/test in CI** (GitHub Actions, GitLab CI) — run linting, tests, type checks before anything touches the server.
2. **Pull-based deploy** (simple setups):
3. `cd /home/deploy/apps/myappgit pull origin mainsource venv/bin/activatepip install -r requirements.txtpython manage.py migratepython manage.py collectstatic --noinputsudo systemctl restart myapp`
4. **Zero-downtime restarts**: Gunicorn supports graceful reload via `kill -HUP <master_pid>`, which restarts workers one at a time without dropping connections.
5. **Blue-green / rolling deploys**: run a second instance on a different port, switch Nginx upstream once healthy, then tear down the old one — avoids any downtime window.

6. Consider **Ansible** for multi-server or repeatable configuration, or move to **Docker + a container orchestrator** (Docker Compose for single-host, Kubernetes/Nomad for fleets) once you outgrow a single systemd-managed box.
-

11. Security Checklist

- ☐ App runs as non-root user
 - ☐ DEBUG=False, no stack traces exposed to users
 - ☐ Secrets in environment variables / vault, not in git
 - ☐ .env file permissions locked to 600
 - ☐ Firewall (ufw) restricts open ports to 22/80/443 only
 - ☐ TLS enabled and auto-renewing
 - ☐ Dependencies scanned for CVEs (pip-audit, safety)
 - ☐ fail2ban or similar guarding SSH
 - ☐ Regular OS patching (unattended-upgrades)
 - ☐ Backups of DB and .env/secrets configured and tested (restore, not just backup)
 - ☐ Rate limiting / WAF at Nginx or CDN layer for public-facing apps
-

12. Optional: Containerized Deployment

If you'd rather not manage systemd/venv directly, a minimal production Dockerfile:

FROM python:3.12-slim

`WORKDIR /app`

`COPY requirements.txt .`

`RUN pip install --no-cache-dir -r requirements.txt`

`COPY . .`

`CMD ["gunicorn", "-c", "gunicorn_conf.py", "src.wsgi:application"]`

Run behind the same Nginx reverse-proxy pattern, or use docker compose to wire app + db + nginx together. This trades systemd/venv complexity for image build/registry complexity — worth it once you have multiple services or need consistent environments across dev/staging/prod.

Quick Reference: Minimal Production Stack

Internet → Nginx (TLS, static files, proxy) → Gunicorn/Uvicorn (app server, systemd-managed) → Application code (venv-isolated) → Database